Phase 07 — VPS and Deployment (Complete Notes)

> *First real deployment: your app, on a public server, behind a real domain, with HTTPS.*

---

1. What is a VPS?

Virtual Private Server — a slice of a physical machine in a datacenter, carved out by virtualization. You get: your own Ubuntu install, root access, a **public IPv4 address** (no NAT problem — Phase 1!), always on.

Hosting spectrum

	Shared hosting	VPS	Dedicated	Cloud (AWS)
What	folder on shared server	virtual machine, root access	whole physical machine	VMs + 200 managed services
Control	none (cPanel)	full	full	full + APIs
Price	$2–5/mo	**$5–15/mo**	$80+/mo	pay-per-use, varies
For	WordPress	★ learning + small production	special needs	scale, teams (Phase 8)

Analogy: shared hosting = renting a bed in a dorm; VPS = your own apartment in a building (shared structure, private space, your keys); dedicated = owning the house; cloud = a hotel chain where you summon rooms on demand.

Providers to use: **Hetzner** (~€4.5/mo, excellent), **Contabo** (cheap, more RAM), DigitalOcean/Vultr (polished, good docs). Any Ubuntu 24.04 LTS option, smallest size, is fine for learning.

2. First 30 minutes on a new VPS — the security ritual

You get an email: IP + root password. Bots start brute-forcing SSH within *minutes* of a server going online (check `auth.log` later — you'll see them). Ritual, in order:

```
ssh root@203.0.113.10                        # 1. first login

apt update && apt upgrade -y                 # 2. patch everything

adduser deploy                               # 3. working user
usermod -aG sudo deploy

# 4. SSH keys (from YOUR LAPTOP)
ssh-copy-id deploy@203.0.113.10
ssh deploy@203.0.113.10                      # verify key login works BEFORE step 5!

sudo nano /etc/ssh/sshd_config               # 5. harden SSH:
#    PermitRootLogin no
#    PasswordAuthentication no
sudo systemctl restart ssh

sudo ufw allow OpenSSH                       # 6. firewall (Phase 4 — SSH first!)
sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
sudo ufw enable

sudo apt install fail2ban -y                 # 7. auto-ban brute-force IPs
sudo timedatectl set-timezone Asia/Dhaka     # 8. sane clocks for logs
```

Result: no root login, no passwords, only ports 22/80/443, brute-forcers banned. This ritual is Assignment material — know it cold.

3. Deploying the stack

Two valid approaches — know both:

A. Classic (no Docker): everything as systemd services

```
sudo apt install nginx postgresql openjdk-21-jre-headless
scp target/app.jar deploy@vps:/opt/shop/          # or build on server via git pull + mvn
# systemd unit (Phase 4), nginx reverse proxy (Phase 5), certbot SSL
```

B. Docker Compose (recommended — same files as Phase 6)

```
sudo apt install docker.io docker-compose-v2
git clone your-repo && cd your-repo
nano .env                                     # real secrets (never in git)
docker compose up -d
```

The compose file you built in Phase 6 runs **unchanged**. That's the whole point of containers.

4. Connecting the domain

At your DNS provider (Phase 2 knowledge):

```
A    example.com      → 203.0.113.10     TTL 300
A    www.example.com  → 203.0.113.10
A    api.example.com  → 203.0.113.10
```

```
dig example.com +short          # wait until it returns your VPS IP
curl http://example.com         # NGINX answers → domain works
```

5. HTTPS with certbot

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com -d api.example.com
sudo certbot renew --dry-run    # auto-renewal is set up via systemd timer
```

(If NGINX runs in Docker: run certbot on the host with webroot mode, or use a companion container — either way understand it's the same HTTP-challenge mechanism: Let's Encrypt calls `http://your-domain/.well-known/acme-challenge/...` and must reach YOUR server, which is why DNS must point at it first.)

Milestone reached

```
Browser → DNS (your A record) → your VPS public IP → ufw → NGINX (TLS) → Spring Boot →
PostgreSQL/Redis
```

Every single hop is now something YOU built and understand.

6. Monitoring a VPS (pragmatic level)

```
htop                        # CPU/RAM live
df -h                       # disk (logs + docker images fill disks!)
docker stats                # per-container resources
journalctl -u nginx --since today -p err
sudo tail -f /var/log/nginx/access.log
```

- **Uptime monitoring** (external!): UptimeRobot / Better Stack free tier pings your site every minute, emails you when it's down. A server cannot report its own death — the watcher must be outside.
- Add a `/actuator/health` endpoint (Spring Boot Actuator) and monitor *that*, not just port 443 — "process up" ≠ "app healthy".

7. Backups

Rule: **a backup you haven't restored is a hope, not a backup.**

```
# database dump (the data is what matters — servers are replaceable)
docker compose exec db pg_dump -U postgres shop | gzip > backup_$(date +%F).sql.gz

# ship it OFF the server (a backup on the dying disk dies too)
scp backup_*.sql.gz you@other-machine:backups/     # or rclone to S3/Backblaze

# automate: crontab -e
0 3 * * * /home/deploy/backup.sh                   # nightly at 03:00
```

Provider snapshots (Hetzner ~€1/mo) are a good second layer: whole-disk restore. Practice the restore once: create a fresh DB and `gunzip -c backup.sql.gz | psql ...`.

8. Simple deployment pipeline (manual → scripted)

`deploy.sh` on the server:

```
#!/bin/bash
set -e                        # stop on first error
cd /home/deploy/shop
git pull origin main
docker compose build api
docker compose up -d api      # replaces only the app container; db/nginx keep running
docker compose ps
curl -f localhost/api/actuator/health   # smoke test — fail loudly if broken
```

Deploy = `ssh vps ./deploy.sh`. (Real CI/CD with GitHub Actions arrives with the final project.)

9. Common mistakes

- Enabling ufw before allowing SSH (locked out); breaking sshd_config without testing key login first (locked out politely).
- Exposing PostgreSQL/Redis ports publicly (the #1 real-world VPS breach; scanners find open 5432/6379 within hours).
- Secrets committed to git; `.env` pushed to a public repo.
- No swap on a 1 GB RAM VPS → the kernel OOM-kills your JVM randomly (`sudo fallocate -l 2G /swapfile ...`).
- Buying an oversized VPS "for the future" — start tiny, resize later.
- Backups on the same disk; backups never tested; no external uptime check ("we found out from customers").

10. Interview questions

1. VPS vs shared hosting vs cloud — trade-offs?
2. Walk through securing a fresh Ubuntu server (the ritual, in order, with reasoning).
3. Why disable SSH password authentication? What does fail2ban add on top?
4. How does Let's Encrypt verify domain ownership? Why must DNS point at the server first?
5. Your VPS died completely. Walk through recovery — what determines your data loss window? (backup frequency = RPO)
6. Why monitor from OUTSIDE the server? Why monitor a /health endpoint instead of just the port?
7. What is the deploy flow for a Docker-based app on a VPS, with a rollback story?

11. LAB — the real thing

1. Rent the cheapest VPS (Hetzner CX22 or similar, Ubuntu 24.04). This costs ~$5 — the best $5 of this course.
2. Perform the entire §2 security ritual. Then `sudo grep "Failed password" /var/log/auth.log | wc -l` after 24h — meet the bots.
3. Deploy your Phase-6 compose stack (§3B).
4. Point a domain (or free subdomain from DuckDNS if you didn't buy one) at it; get HTTPS via certbot.
5. Set up UptimeRobot on your /health URL; then `docker compose stop api` and wait for the alert email. Start it again.
6. Configure the nightly pg_dump cron; next day, download the dump and restore it into a local PostgreSQL to prove it works.

12. ASSIGNMENT 07 (submit to Claude)

1. Paste (secrets redacted): `ufw status verbose`, the modified sshd_config lines, and `docker compose ps` from your VPS.
2. Paste `curl -I https://your-domain` output showing 200 + your TLS setup working.
3. How many failed SSH attempts did auth.log record in your first 24 hours? What username did bots try most?
4. Write your disaster-recovery runbook: VPS is gone, you have last night's dump + your git repo. Numbered steps, target time, expected data loss.
5. Explain why `set -e` and the final `curl -f` health check matter in deploy.sh — what failure mode does each prevent?